

Documentación de Validación y Control de Calidad

Reto 3 — Diseño y Optimización de Flujos de Trabajo ETL Avanzados

Autor: Judit Giravent

Herramienta ETL: Apache NiFi 1.28.1

Fecha: Septiembre 2026

1. Introducción

Este documento describe los **procesos de validación** aplicados en cada fase del pipeline ETL para garantizar la **integridad, precisión y consistencia** de los datos procesados, así como los **ajustes realizados** durante la implementación.

2. Validación por fase

2.1 Fase de Extracción

Rama CSV — Ventas Retail

Validación	Método	Resultado
Existencia del archivo	Verificación manual en <code>data/raw/csv/</code>	 <code>onlineRetail_10k.csv</code> presente
Lectura correcta	Contador del procesador <code>getFile</code>	 1 FlowFile procesado
Conversión CSV→JSON	Inspección manual del JSON generado	 Estructura JSON válida

Rama API — Cripto

Validación	Método	Resultado
Existencia del JSON	Verificación manual en <code>data/raw/api/</code>	 <code>cripto_precios.json</code> presente
Lectura correcta	Contador de <code>getFile</code>	 1 FlowFile procesado
Extracción de valores	<code>EvaluateJsonPath</code> (In: 1, Out: 1 por <code>matched</code>)	 6 valores extraídos

Rama SQLite — Clientes

Validación	Método	Resultado
Conexión a la BD	<code>DBCConnectionPool</code> en estado <code>Enabled</code>	✔ Conectado
Existencia de la tabla	Consulta <code>SELECT name FROM sqlite_master WHERE type='table'</code>	✔ Tabla <code>clientes</code> presente
Ejecución del SELECT	<code>ExecuteSQLRecord</code> (In: 1, Out: 1)	✔ 2.000 registros procesados
Tamaño del JSON	Verificación manual	✔ 438 KB

2.2 Fase de Transformación

Validación	Método	Resultado
Formato JSON correcto	Inspección con VS Code	✔ Estructura válida
Atributos extraídos	Verificación en <code>AttributesToJson</code>	✔ Todos los campos presentes
Unicidad de registros	Conteo manual	✔ Sin duplicados

2.3 Fase de Carga

Validación	Método	Resultado
Unión de las 3 ramas	<code>MergeContent</code> (In: 3, Out: 1)	✔ 3 FlowFiles unidos
Guardado del dataset final	<code>PutFile</code> (In: 1, Out: 1)	✔ Archivo creado
Existencia del archivo	<code>ls data/final/</code>	✔ <code>dataset_final.json</code> (2.3 MB)
Contenido del archivo	Inspección manual	✔ 3 bloques concatenados

3. Métricas de rendimiento

Rama	Tiempo de ejecución	Tamaño del resultado
CSV (10.000 filas)	~1-2 segundos	9.15 MB
API (4 criptos)	~0.5 segundos	157 bytes
SQLite (2.000 clientes)	~1 segundo	438 KB
Unión final	~1 segundo	2.3 MB
Total pipeline	~5-10 segundos	2.3 MB

4. Ajustes y correcciones realizados

Fase	Problema	Solución aplicada
Extracción API	<code>InvokeHTTP</code> devolvía 403 por User-Agent rechazado	Se descargó el JSON previamente con PowerShell y se usó <code>GetFile</code>
Extracción SQLite	El <code>ExecuteSQLRecord</code> no tenía la query SQL configurada	Se configuró <code>SQL select query = SELECT * FROM clientes</code>
Fase de unión	El <code>MergeContent</code> no mostraba las propiedades necesarias	Se cambió <code>Merge Strategy</code> a <code>Bin-Packing Algorithm</code>
Fase de carga	El <code>PutFile</code> daba <code>InvalidPathException</code> por caracteres inválidos	Se renombró con un nombre sin caracteres problemáticos
Ejecución de procesos de origen	Los procesadores de origen no se ejecutaban	Se añadió <code>GenerateFlowFile</code> como disparador en la rama SQLite

5. Resultados de la validación de calidad

5.1 Integridad referencial

Verificación	Método	Resultado
Total de registros de ventas	Conteo en el JSON	✓ 10.000
Total de criptos	Conteo en el JSON	✓ 4
Total de clientes	Conteo en el JSON	✓ 2.000

5.2 Consistencia de tipos

Campo	Tipo esperado	Tipo detectado	Resultado
<code>InvoiceNo</code>	String	String	✓
<code>Quantity</code>	Integer	Integer	✓
<code>UnitPrice</code>	Float	Float	✓
<code>cliente_id</code>	String	String	✓
<code>edad</code>	Integer	Integer	✓
<code>ingresos_anuales</code>	Integer	Integer	✓
<code>btc_usd</code>	Number	String (JSON)	✓ (JSON siempre string en JSONPath)

5.3 Estructura del dataset final

```
[  
  [ {...ventas...}, {...ventas...}, ... ],      ← Bloque 1: 10.000 ventas  
  { "bitcoin": {...}, "ethereum": {...}, ... }, ← Bloque 2: cripto  
  [ {...clientes...}, {...clientes...}, ... ]    ← Bloque 3: 2.000 clientes  
]
```

6. Conclusiones

- El pipeline ETL se ha ejecutado **sin errores** en las 3 ramas.
- El **dataset final** contiene los 3 bloques unificados.
- Todos los **ajustes documentados** se han validado y funcionan correctamente.
- El flujo es **reproducible** y puede ejecutarse de nuevo con los mismos datos.
- Los **tiempos de ejecución** son aceptables (< 10 segundos para ~10.000 registros).

7. Anexos

- **Flujo NiFi:** `Proyecto_ETL_Completo_Reto3_JuditGiravent.xml`
- **Dataset final:** `data/final/dataset_final.json`
- **Capturas:** `outputs/screenshots/`
- **Informe del proceso:** `reports/informe_proceso_etl.md`